

CS432: Databases

Indexing and Hashing

Instructor
Yogesh K. Meena

Lecture no.
19

CSE, IIT Gandhinagar
April 1, 2026

An Index

- An **index** for a file in a database system is similar to index in the textbook.
- To retrieve a student record given an ID, the database system would look up an index to find on which disk block the corresponding record resides.
- Index sorting reduces the search time.

Two Types

- **Ordered indices:** Based on a sorted ordering of the values.
- **Hash indices:** Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a *hash function*.

Evaluating different indexing techniques

- **Access types:** Finding records given specified attribute value, or range of values.
- **Access time:** Time to find a particular data item, or set of items.
- **Insertion time:** Time to insert a new data item + update the index structure
- **Deletion time:** Time to delete a new data item + update the index structure
- **Space overhead:** Space occupied by an index structure

Ordered Indices

- An attribute or set of attributes used to look up records in a file is called a **search key**.
- An ordered index stores the values of the search keys in sorted order.
- If the file containing the records is sequentially ordered:
 - **Clustering index:** search key also defines the sequential order of the file. **Primary indices**
 - **Nonclustering indices:** search key specifies an order different from the sequential order of the file. **Secondary indices**

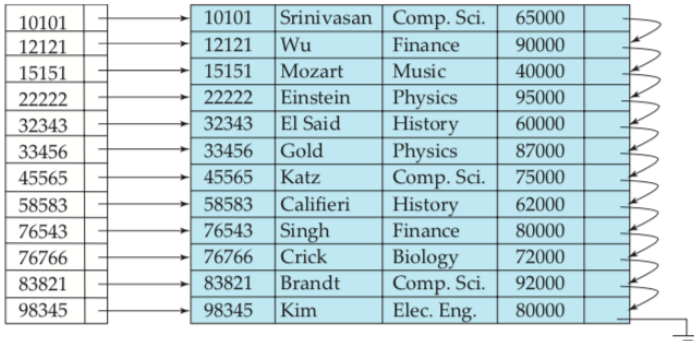
Sequential file

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



Dense index

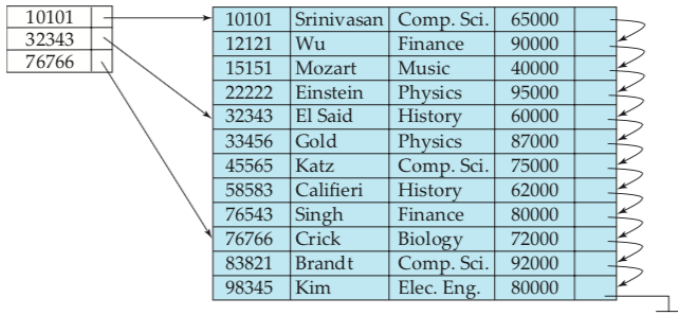
An index entry appears for every search-key value in the file.



What happens to records with the same search-key value?

Sparse index

An index entry appears for only some of the search-key values



- Search the index entry with the largest search-key value that is less than or equal to the queried search-key value.
- Start at the record pointed to by that index entry, and follow the pointers in the file.

Trade-off b/w access time and space overhead

- Dense indices are faster.
- sparse indices require less space.

A good compromise = **sparse index with one index entry per block**

Multilevel Indices

- A relation with 100,000,000 tuples.
- 100 index entries fit on a 4KB block.



Multilevel Indices

- A relation with 100,000,000 tuples.
- 100 index entries fit on a 4KB block.
- 1,000,000 blocks are required



Multilevel Indices

- A relation with 100,000,000 tuples.
- 100 index entries fit on a 4KB block.
- 1,000,000 blocks are required
- 4GB of space is required.
- Search for an entry in the index requires several disk-block reads.

Multilevel Indices

- A relation with 100,000,000 tuples.
- 100 index entries fit on a 4KB block.
- 1,000,000 blocks are required
- 4GB of space is required.
- Search for an entry in the index requires several disk-block reads.

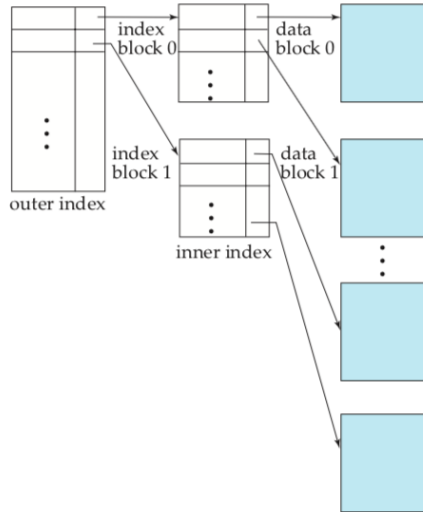
Binary Search

If index occupies b blocks, binary search requires $\log_2(b)$ blocks to be read.

Example:

On a disk system where a block read takes on average 10 milliseconds, 10,000-block index will take 140 milliseconds.

Multilevel Indices



Index Insertion (single-level indices)

Dense indices

- If the search-key value does not appear in the index, the system inserts an index entry with the search-key value in the index at the appropriate position.
- Otherwise the following actions are taken:
 - If the index entry stores pointers to all records with the same search-key value, the system adds a pointer to the new record in the index entry.
 - Otherwise, the index entry stores a pointer to only the first record with the search-key value. The system then places the record being inserted after the other records with the same search-key values.

Index Insertion (single-level indices)

Sparse indices

Assuming that index stores an entry for each block.

- If the system creates a new block, it inserts the first search-key value appearing in the new block into the index.
- Otherwise if the new record has the least search-key value in its block, the system updates the index entry pointing to the block.
- if not, the system makes no change to the index.

Index Deletion (single-level indices)

Dense indices

- If the deleted record was the only record with its particular search-key value, then the system deletes the corresponding index entry from the index.
- Otherwise the following actions are taken:
 - If the index entry stores pointers to all records with the same search-key value, the system deletes the pointer to the deleted record from the index entry.
 - Otherwise, the index entry stores a pointer to only the first record with the search-key value. In this case, if the deleted record was the first record with the search-key value, the system updates the index entry to point to the next record.

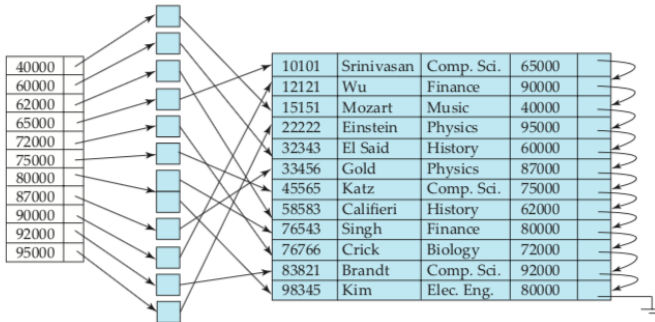
Index Deletion (single-level indices)

Sparse indices

- If the index does not contain an index entry with the search-key value of the deleted record, nothing needs to be done to the index.
- Otherwise the following actions are taken:
 - If the deleted record was the only record with its search key, the system replaces the corresponding index record with an index record for the next search-key value (in search-key order). If the next search-key value already has an index entry, the entry is deleted instead of being replaced.
 - Otherwise, if the index entry for the search-key value points to the record being deleted, the system updates the index entry to point to the next record with the same search-key value.

Secondary Indices

- It must be dense, with an index entry for every search-key value, and a pointer to every record in the file.
- If the search key of a secondary index is not a candidate key, it is not enough to point to just the first record with each search-key value.



B^+ –Tree Index Files

Node of a B^+ –Tree

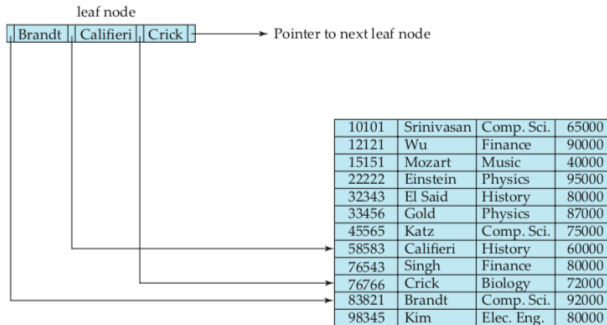
P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

B^+ –Tree Index Files

Node of a B^+ –Tree

P_1	K_1	P_2	...	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	-----	-----------	-----------	-------

A leaf node (n=4)



Properties of leaf nodes

- Each leaf can hold up to $n-1$ values.
- Each leaf should contain atleast $\frac{n-1}{2}$ values.
- if L_i and L_j are leaf nodes and $i < j$, then every search-key value in L_i is less than or equal to every search-key value in L_j
- P_n is used to chain together the leaf nodes in search-key order.

Properties of nonleaf nodes

- All pointers are pointers to tree nodes.
- A nonleaf node may hold up to n pointers, and must hold at least $\frac{n}{2}$ pointers.

Some more points to remember

Understanding the non-leaf pointers

Consider a nonleaf node containing m pointers ($m \leq n$).

- Pointer P_i , where $i = 2, \dots, m - 1$, points to the subtree that contains search-key values less than K_i and greater than or equal to K_{i-1} .
- Pointer P_m points to the part of the subtree that contains those key values greater than or equal to K_{m-1}
- Pointer P_1 points to the part of the subtree that contains those search-key values less than K_1 .

Understanding the pointers at root node

- The root node can hold fewer than $\frac{n}{2}$ pointers.
- It must hold at least two pointers.

An Example

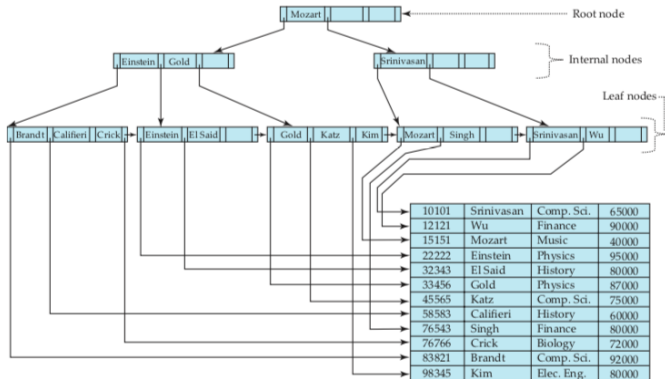


Figura: B^+ -tree for instructor file ($n = 4$)

Queries on B^+ -Tree

```
function find(value  $V$ )
/* Returns leaf node  $C$  and index  $i$  such that  $C.P_i$  points to first record
* with search key value  $V$  */
  Set  $C$  = root node
  while ( $C$  is not a leaf node) begin
    Let  $i$  = smallest number such that  $V \leq C.K_i$ 
    if there is no such number  $i$  then begin
      Let  $P_m$  = last non-null pointer in the node
      Set  $C = C.P_m$ 
    end
    else if ( $V = C.K_i$ )
      then Set  $C = C.P_{i+1}$ 
    else  $C = C.P_i$  /*  $V < C.K_i$  */
  end
/*  $C$  is a leaf node */
  Let  $i$  be the least value such that  $K_i = V$ 
  if there is such a value  $i$ 
    then return ( $C, i$ )
    else return null ; /* No record with key value  $V$  exists */
```

Queries on B^+ –Tree

```
procedure printAll(value V)  
/* prints all records with search key value V */  
  Set done = false;  
  Set (L, i) = find(V);  
  if ((L, i) is null) return  
  repeat  
    repeat  
      Print record pointed to by L.Pi  
      Set i = i + 1  
    until (i > number of keys in L or L.Ki > V)  
    if (i > number of keys in L)  
      then L = L.Pn  
      else Set done = true;  
  until (done or L is null)
```

Output records within a given range

`printRange(L,U)`



Complexity of querying



Complexity of querying

Time complexity: $\log_{\frac{n}{2}}(N)$

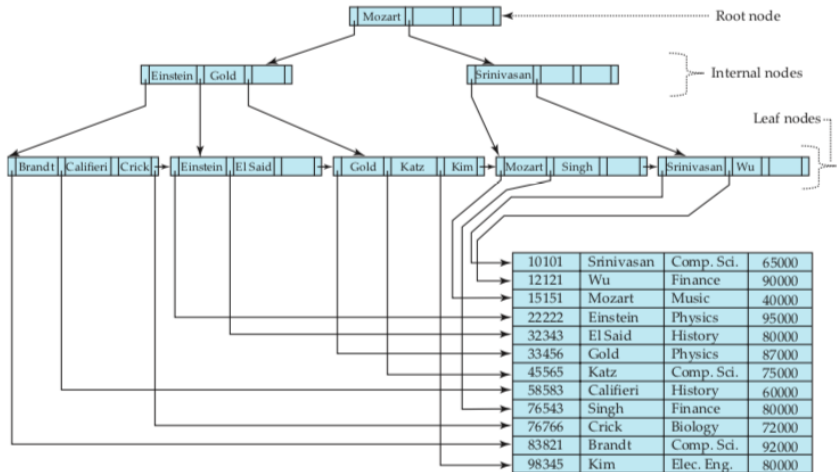


Updates on B^+ –Tree

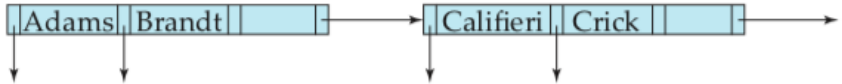
- Insertion.
- Deletion.



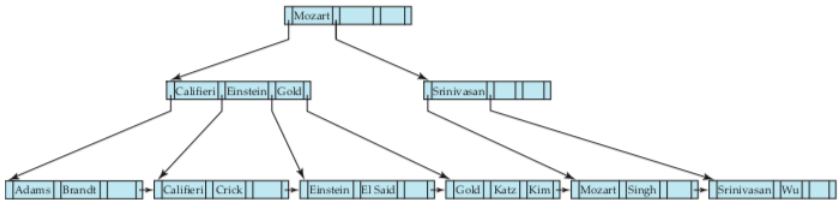
Insertion on B^+ -Tree



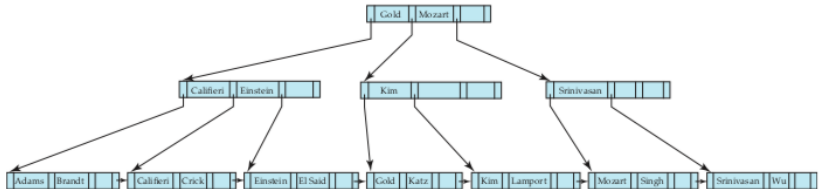
Insertion on B^+ -Tree



Insertion on B^+ -Tree

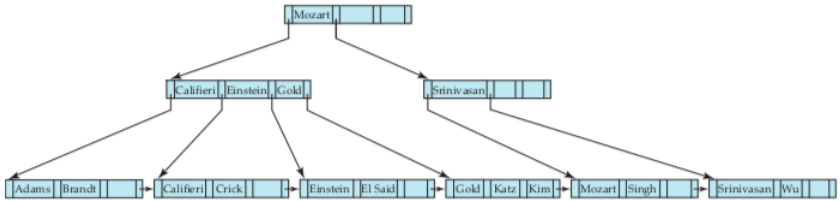


Insertion on B^+ -Tree



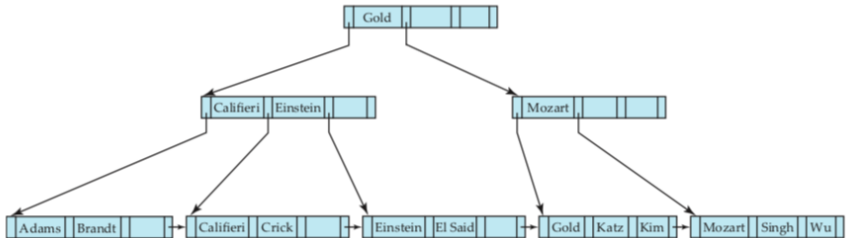
Insert the record **Lampport**

Deletion on B^+ -Tree



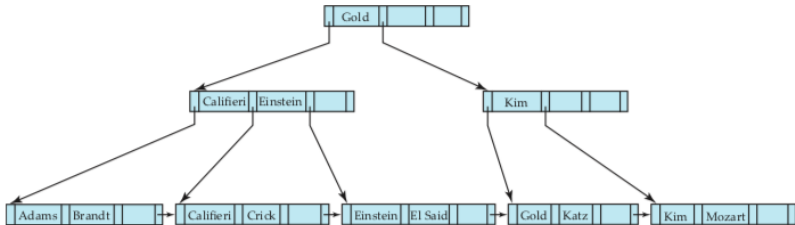
Delete the record **Srinivasan**

Deletion on B^+ -Tree



After deleting the record **Srinivasan**

Deletion on B^+ -Tree



After deleting the record **Singh** and **Wu**

Multiple-Key Access

Find all instructors in the Finance department with salary equal to \$80,000.

```
select ID  
from instructor  
where dept_name = 'Finance' and salary = 80000;
```

Three possible strategies?

Indices on Multiple Keys

Leverage composite search keys (dept_name,salary) in B^+ -Tree.
Name concatenated with the instructor salary.

```
select ID  
from instructor  
where dept_name = 'Finance' and salary = 80000;
```

Static Hashing

Disadvantage of sequential file organization:

- Must access an index structure to locate data.
- Must use binary search.

Results in more I/O operations



Static Hashing

Disadvantage of sequential file organization:

- Must access an index structure to locate data.
- Must use binary search.

Results in more I/O operations

Use hashing to avoid accessing an index structure.

Buckets and Hash Function

Bucket

- Denotes a unit of storage that can store one or more records.
- Typically a disk block, but can be larger or smaller.

Buckets and Hash Function

Bucket

- Denotes a unit of storage that can store one or more records.
- Typically a disk block, but can be larger or smaller.

Hash function (h)

- Let K denote the set of all search-key values.
- Let B denote the set of all bucket addresses.

h is a function from K to B .

Buckets and Hash Function

Bucket

- Denotes a unit of storage that can store one or more records.
- Typically a disk block, but can be larger or smaller.

Hash function (h)

- Let K denote the set of all search-key values.
- Let B denote the set of all bucket addresses.

h is a function from K to B .

Given a search-key value K_i , we compute $h(K_i)$, then search the bucket with that address.

Hash Functions

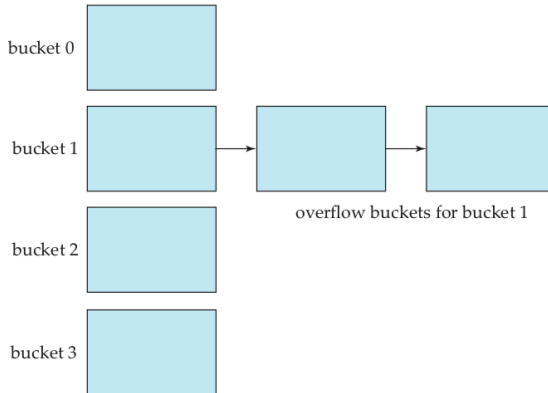
- **Uniform:** Assign each bucket the same number of search-key values from the set of all possible search-key values.
- **Random:** On an average, each bucket will have nearly the same number of values assigned to it.

Handling of Bucket Overflows

- **Insufficient buckets:** The number of buckets, n_B , must be chosen such that $n_B > \frac{n_r}{f_r}$.
- **Skew:** Some buckets are assigned more records than are others, so a bucket may overflow even when other buckets still have space.
 - Multiple records may have the same search key.
 - The chosen hash function may result in nonuniform distribution of search keys.

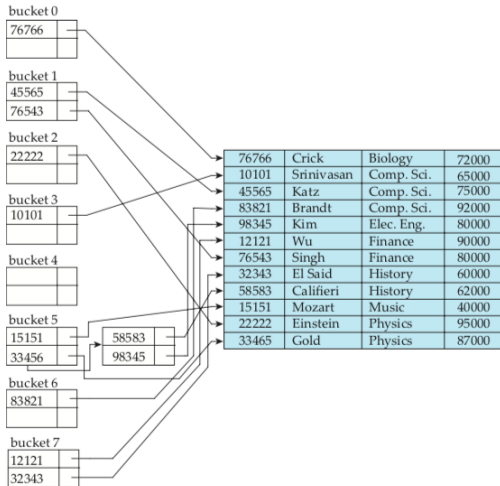
Standard n_B assignment: $n_B = \frac{n_r}{f_r} * (1 + d)$, d is a fudge factor, typically around 0.2.

Overflow Buckets



Open or closed hashing.

Hash index



Dynamic Hashing

- In static hashing, we fix the set B of bucket addresses.

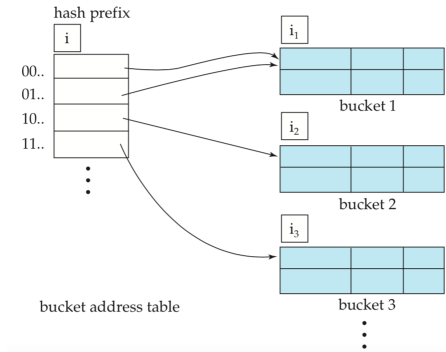


Dynamic Hashing

- In static hashing, we fix the set B of bucket addresses.
- Dynamic hashing allow the hash function to be modified dynamically.

Extendable Hashing

Extendable Hash Structure



i : Number of bits required to find the correct bucket.

i_j : Length of the common hash prefix

Instructor file

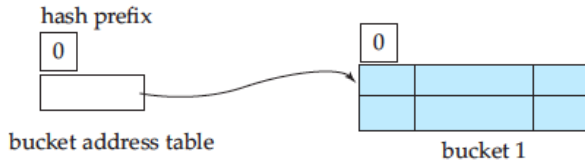
10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



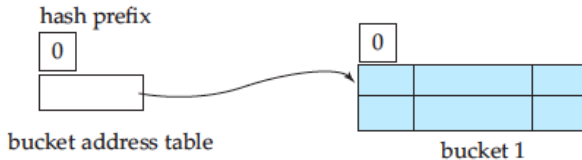
Department Hash values

<i>dept_name</i>	<i>h(dept_name)</i>
Biology	0010 1101 1111 1011 0010 1100 0011 0000
Comp. Sci.	1111 0001 0010 0100 1001 0011 0110 1101
Elec. Eng.	0100 0011 1010 1100 1100 0110 1101 1111
Finance	1010 0011 1010 0000 1100 0110 1001 1111
History	1100 0111 1110 1101 1011 1111 0011 1010
Music	0011 0101 1010 0110 1100 1001 1110 1011
Physics	1001 1000 0011 1111 1001 1100 0000 0001

Initial extendable hash structure

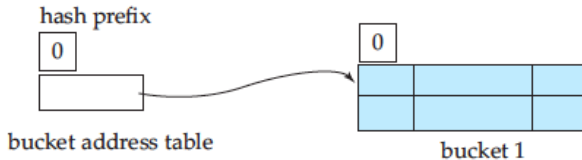


Initial extendable hash structure



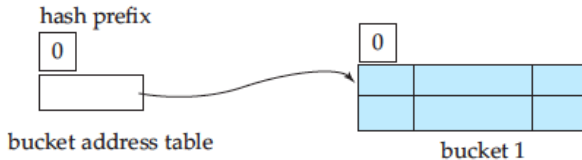
1. Add a new record: (10101, Srinivasan, Comp. Sci., 65000).

Initial extendable hash structure



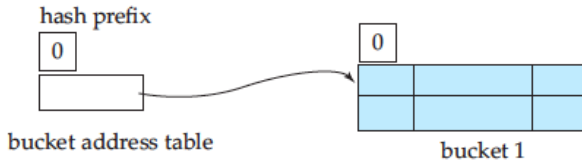
1. Add a new record: (10101, Srinivasan, Comp. Sci., 65000).
2. Add one more record: (12121, Wu, Finance, 90000).

Initial extendable hash structure



1. Add a new record: (10101, Srinivasan, Comp. Sci., 65000).
2. Add one more record: (12121, Wu, Finance, 90000).
3. Add (15151, Mozart, Music, 40000)

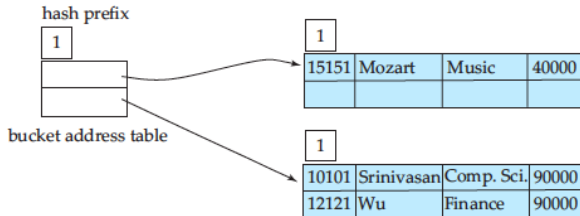
Initial extendable hash structure



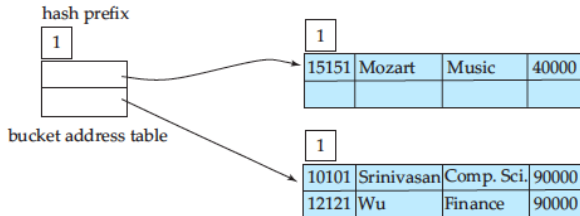
1. Add a new record: (10101, Srinivasan, Comp. Sci., 65000).
2. Add one more record: (12121, Wu, Finance, 90000).
3. Add (15151, Mozart, Music, 40000)

Need to split the bucket

Splitting a bucket

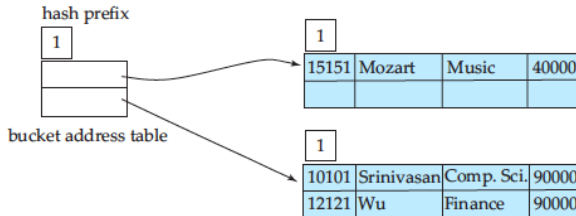


Splitting a bucket



4. Insert (22222, Einstein, Physics, 95000).

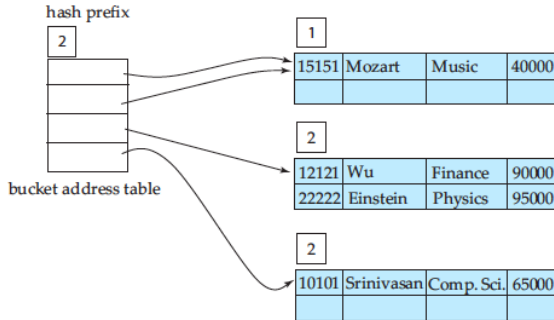
Splitting a bucket



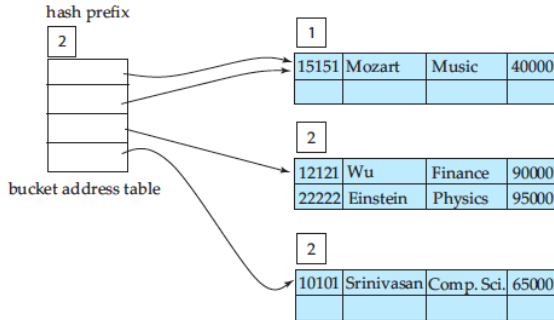
4. Insert (22222, Einstein, Physics, 95000).

Again need to split the bucket

Splitting a bucket

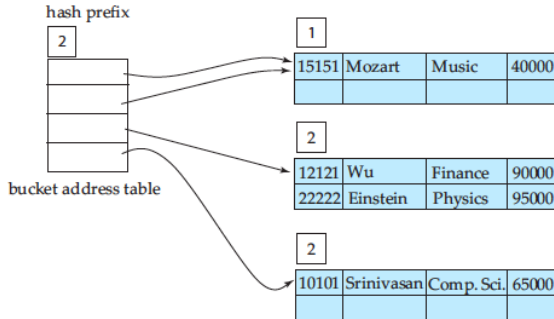


Splitting a bucket



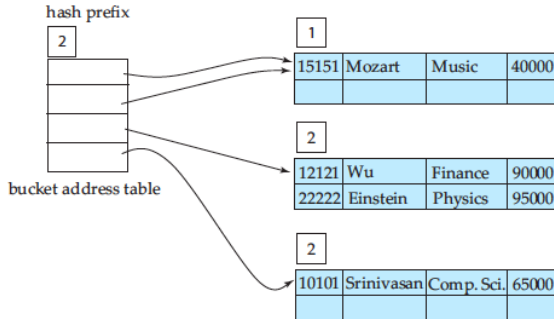
5. Insert (32343, El Said, History, 60000).

Splitting a bucket



5. Insert (32343, El Said, History, 60000).
6. Insert (33456, Gold, Physics, 87000).

Splitting a bucket

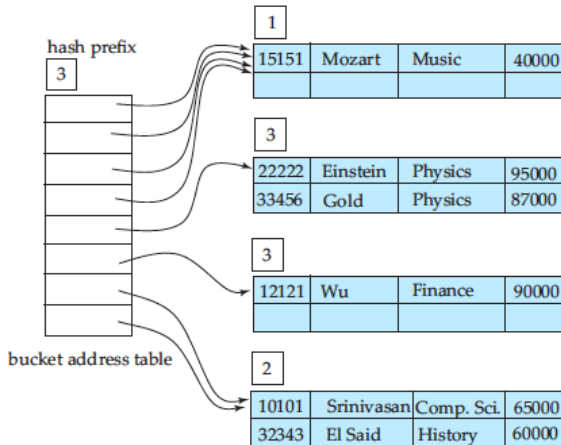


5. Insert (32343, El Said, History, 60000).

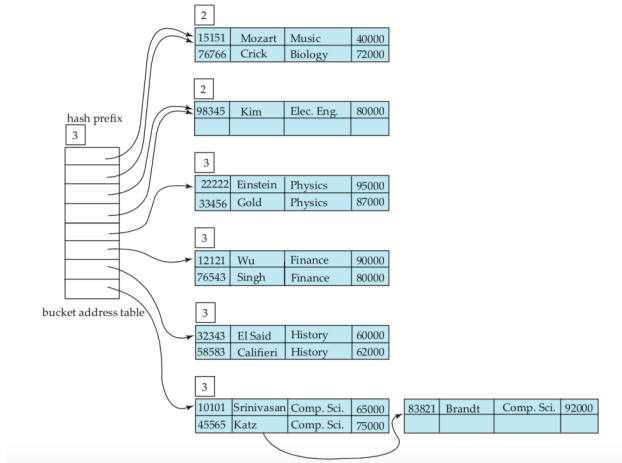
6. Insert (33456, Gold, Physics, 87000).

Again need to split the bucket

Splitting a bucket



Extendable hash structure for the instructor file



Static Hashing versus Dynamic Hashing?

- Performance does not degrade as the file grows.
- In extendable hashing lookup involves an additional level of indirection.

Ordered Indexing versus Hashing?

select $A_1, A_2, \dots, A_n$
from r
where $A_i = c;$

select $A_1, A_2, \dots, A_n$
from r
where $A_i \leq c_2$ **and** $A_i \geq c_1;$

Bitmap Indices

A bitmap is simply an array of bits.

record number	ID	gender	income_level
0	76766	m	L1
1	22222	f	L2
2	12121	f	L1
3	15151	m	L4
4	58583	f	L3

Bitmaps for gender

m

10010

f

01101

Bitmaps for income_level

L1

10100

L2

01000

L3

00001

L4

00010

L5

00000

How to handle deletions?

Acknowledgments/Contributions

- Some of the images utilized in these slides are subject to copyright - Abraham Silberschatz, Henry Korth, and S. Sudarshan. Database System Concepts. 6th Edition, McGraw-Hill Education
- Contributor 2024-25, 2025-26 - Yogesh K. Meena, IIT Gandhinagar